Assignment_5_Data_624

Mubashira Qari

2026-03-03

Table of Contents

8.1. Consider the the number of pigs slaughtered in Victoria, available in the aus_livestock dataset.

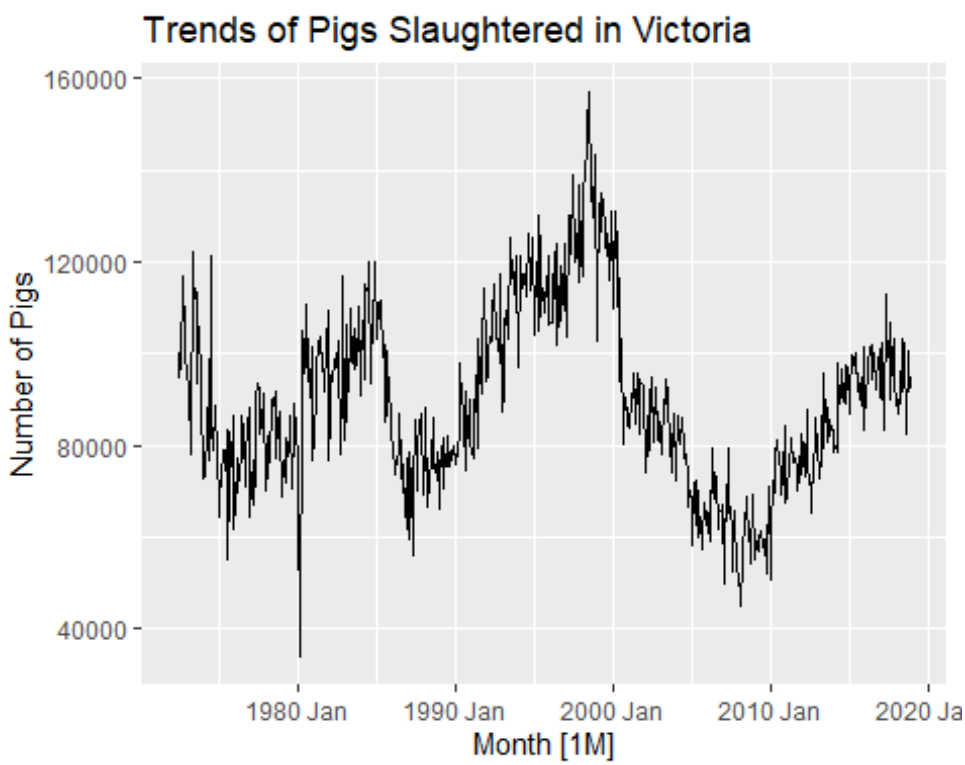
a.Use the ETS() function to estimate the equivalent model for simple exponential smoothing. Find the optimal values of α and ℓ0 , and generate forecasts for the next four months.

```
library(fpp3)

## Registered S3 method overwritten by 'tsibble':
##   method               from
##   as_tibble.grouped_df dplyr

## ── Attaching packages ──────────────────────────────────────────── fpp3
1.0.2 ──

## ✓ tibble      3.2.1     ✓ tsibble     1.1.6
## ✓ dplyr       1.1.4     ✓ tsibbledata 0.4.1
## ✓ tidyr       1.3.1     ✓ feasts      0.4.2
## ✓ lubridate   1.9.4     ✓ fable       0.5.0
## ✓ ggplot2     4.0.0

## Warning: package 'dplyr' was built under R version 4.3.3

## Warning: package 'lubridate' was built under R version 4.3.3

## Warning: package 'tsibble' was built under R version 4.3.3

## Warning: package 'tsibbledata' was built under R version 4.3.3

## ── Conflicts ──────────────────────────────────────────────────────
fpp3_conflicts ──
## ✗ lubridate::date()    masks base::date()
## ✗ dplyr::filter()      masks stats::filter()
## ✗ tsibble::intersect() masks base::intersect()
## ✗ tsibble::interval()  masks lubridate::interval()
## ✗ dplyr::lag()         masks stats::lag()
## ✗ tsibble::setdiff()   masks base::setdiff()
## ✗ tsibble::union()     masks base::union()
```

```
# View(aus_livestock)
```

Filtering & Visualizing the data for Pigs in Victoria

```
pigs_vic <- aus_livestock |>
  filter(Animal == "Pigs", State == "Victoria")

pigs_vic |>
  autoplot(Count) +
  labs(title = "Trends of Pigs Slaughtered in Victoria", y = "Number of
Pigs")
```



Fitting and Estimating the ETS Model (SES)

ETS(A,N,N) model

To perform Simple Exponential Smoothing within the ETS framework, we use error("A"), trend("N"), and season("N")

```
aus_fit <- pigs_vic |>
  model(ANN = ETS(Count ~ error("A") + trend("N") + season("N")))

report(aus_fit)

## Series: Count
## Model: ETS(A,N,N)
##   Smoothing parameters:
```

```
##     alpha = 0.3221247
##
##   Initial states:
##      l[0]
##  100646.6
##
##   sigma^2:  87480760
##
##      AIC     AICc      BIC
## 13737.10 13737.14 13750.07
```

Interpretation

This model assumes the data has an additive error structure but no trend and no seasonality

Smoothing Parameter (Alpha): A moderate alpha value suggests the model places roughly 32% weight on the most recent observation and 68% weight on older data. It produces a balanced forecast that doesn't overreact to monthly fluctuations.

Initial state: This represents the estimated baseline level of pig slaughters at the very start of the time series.

Residual Variance: The value of 87,480,760 indicates the variability of the series.

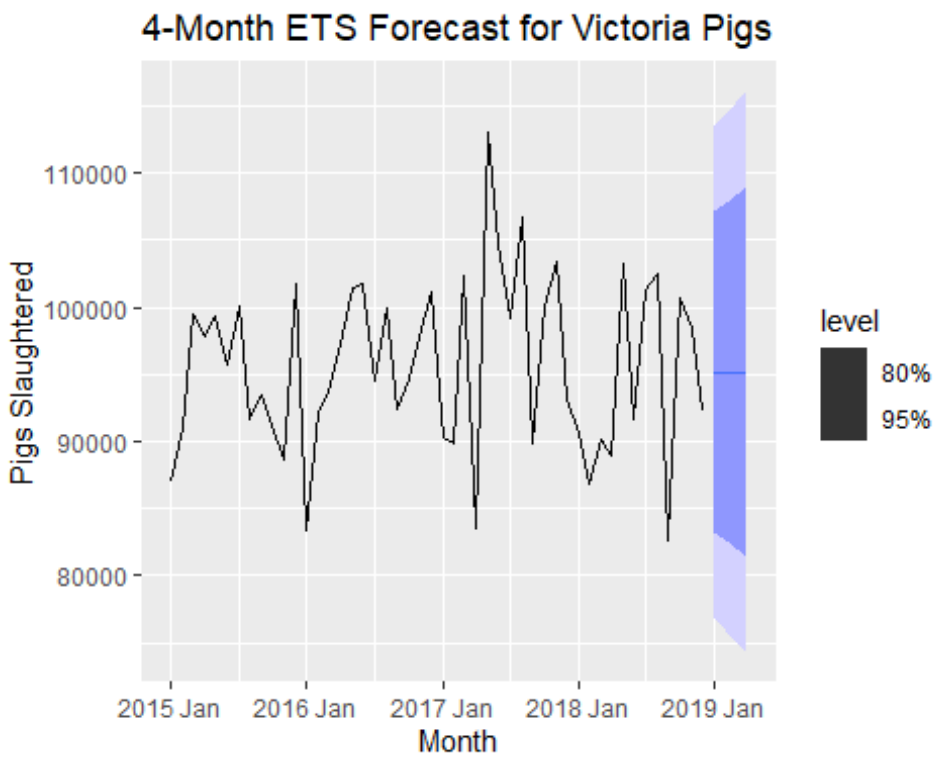
Since there is no trend component, the forecast for the next four months will be a flat horizontal line based on the last estimated level.

Generating Forecasts for 4 Months

```r
aus_forecast <- aus_fit |>
  forecast(h = 4)

# View the forecast table
# aus_forecast
```

Visualize the Results

```r
aus_forecast |>
  autoplot(pigs_vic |>
  filter(Month >= yearmonth("2015 Jan"))) +
  labs(title = "4-Month ETS Forecast for Victoria Pigs",
       y = "Pigs Slaughtered")
```



b. Compute a 95% prediction interval for the first forecast using ^y±1.96s where s is the standard deviation of the residuals. Compare your interval with the interval produced by R.

Extracting the residuals and calculate their standard deviation

```
res_std <- augment(aus_fit) |>
  pull(.resid) |>
  sd(na.rm = TRUE)

# Getting the first forecast value (Point Forecast)
first_fc <- aus_forecast |> filter(row_number() == 1)
y_hat <- first_fc |> pull(.mean)

#Calculating the manual boundaries
lower_manual <- y_hat - 1.96 * res_std
upper_manual <- y_hat + 1.96 * res_std

cat("Manual 95% PI: [", lower_manual, ", ", upper_manual, "]\n")

## Manual 95% PI: [ 76871.01 ,  113502.1 ]
```

Extracting R's Prediction Interval

```
# Extract R's calculated interval
r_interval <- aus_forecast |>
  hilo(level = 95) |>
  filter(row_number() == 1) |>
```

```
  unpack_hilo("95%")

cat("R's 95% PI: [", r_interval$`95%_lower`, ", ", r_interval$`95%_upper`,
"]\n")

## R's 95% PI: [ 76854.79 ,  113518.3 ]
```

Interpretation:

Alpha: The model is stable, favoring long-term history over recent fluctuations.

Point Forecast: The forecast is a flat line at 95,186.56 because the model assumes no trend.

Initial State: The series is estimated to have started at a baseline level of 100,647.

Interval Match: Your manual calculation and R's results are nearly identical for the first month.

Precision Gap: Small differences exist because R uses a precise multiplier and optimized error values.

While the prediction is flat, the fan of uncertainty widens the further you look ahead.

8.5 Data set global_economy contains the annual Exports from many countries. Select one country to analyse.

a. Plot the Exports series and discuss the main features of the data.

Filtering for Australia and plotting

```
library(fpp3)

# Filtering for Australia and plotting
aus_exports <- global_economy |>
  filter(Country == "Australia")
aus_exports

## # A tsibble: 58 x 9 [1Y]
## # Key:       Country [1]
##    Country   Code   Year          GDP Growth   CPI Imports Exports
Population
##    <fct>     <fct> <dbl>        <dbl>  <dbl> <dbl>   <dbl>   <dbl>
<dbl>
##  1 Australia AUS    1960 18573188487.  NA     7.96    14.1    13.0
10276477
##  2 Australia AUS    1961 19648336880.   2.49  8.14    15.0    12.4
10483000
##  3 Australia AUS    1962 19888005376.   1.30  8.12    12.6    13.9
10742000
##  4 Australia AUS    1963 21501847911.   6.21  8.17    13.8    13.0
```

```
10950000
##  5 Australia AUS    1964 23758539590.   6.98  8.40    13.8    14.9
11167000
##  6 Australia AUS    1965 25931235301.   5.98  8.69    15.3    13.2
11388000
##  7 Australia AUS    1966 27261731437.   2.38  8.98    15.1    12.9
11651000
##  8 Australia AUS    1967 30389741292.   6.30  9.29    13.9    12.9
11799000
##  9 Australia AUS    1968 32657632434.   5.10  9.52    14.5    12.3
12009000
## 10 Australia AUS    1969 36620002240.   7.04  9.83    13.3    12.0
12263000
## # ℹ 48 more rows
```

```
aus_exports |>
  autoplot(Exports) +
  labs(title = "Annual Exports for Australia",
       y = "% of GDP", x = "Year")
```

Overall Trend: The data shows a clear, long-term upward trajectory, indicating exports are taking up a larger share of the economy over time.

Cyclicality: Visible cycles appear throughout the decades

Seasonality: There is no seasonality to analyze because the data is recorded on an annual basis.

Structural Shift: A sharp acceleration in growth is evident starting in the 1980s and peaking after 2000.

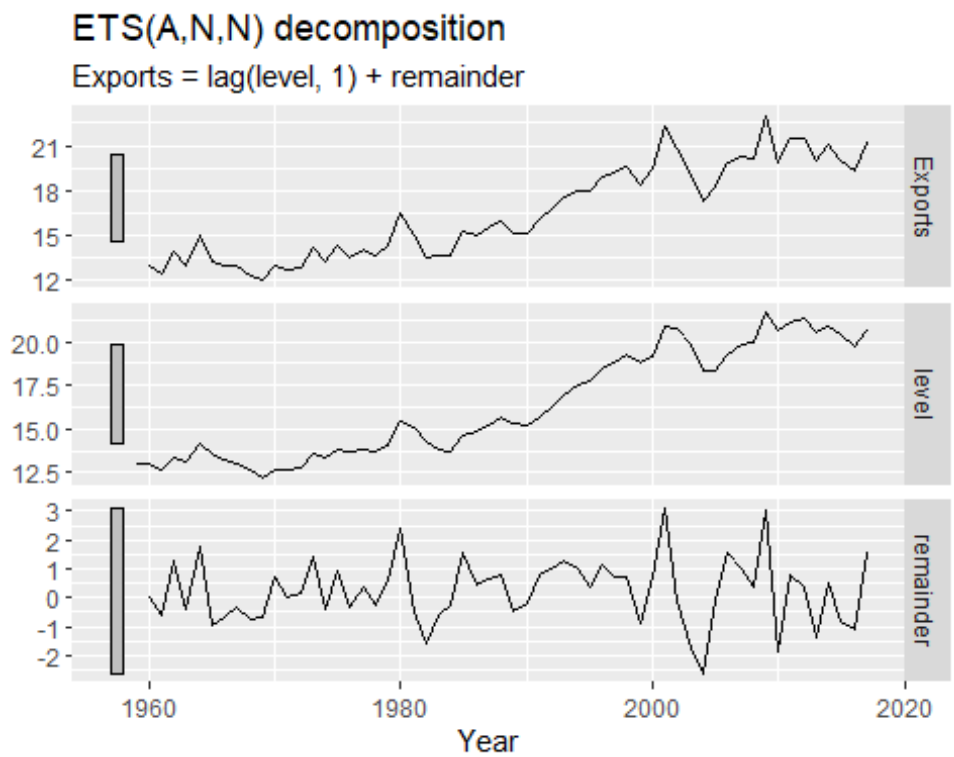
b. Use an ETS(A,N,N) model to forecast the series, and plot the forecasts.

```r
library(fpp3)
# Filtering the data for Australia
aus_exports <- global_economy |>
  filter(Country == "Australia")

# Fitting the ETS(A,N,N) model
aus_exports_fit <- aus_exports |>
  model(ANN = ETS(Exports ~ error("A") + trend("N") + season("N")))

report(aus_exports_fit)
```

```
## Series: Exports
## Model: ETS(A,N,N)
##   Smoothing parameters:
##     alpha = 0.5659948
##
##   Initial states:
##      l[0]
##  12.98943
##
##   sigma^2:  1.3621
##
##      AIC     AICc      BIC
## 257.3943 257.8387 263.5756
```

```r
components(aus_exports_fit) |> autoplot()
```

```
## Warning: Removed 1 row containing missing values or values outside the
scale range
## (`geom_line()`).
```



Now we left Join the fitted values to the components

```
components(aus_exports_fit) |>
 left_join(fitted(aus_exports_fit), by = c("Country", ".model", "Year"))
```

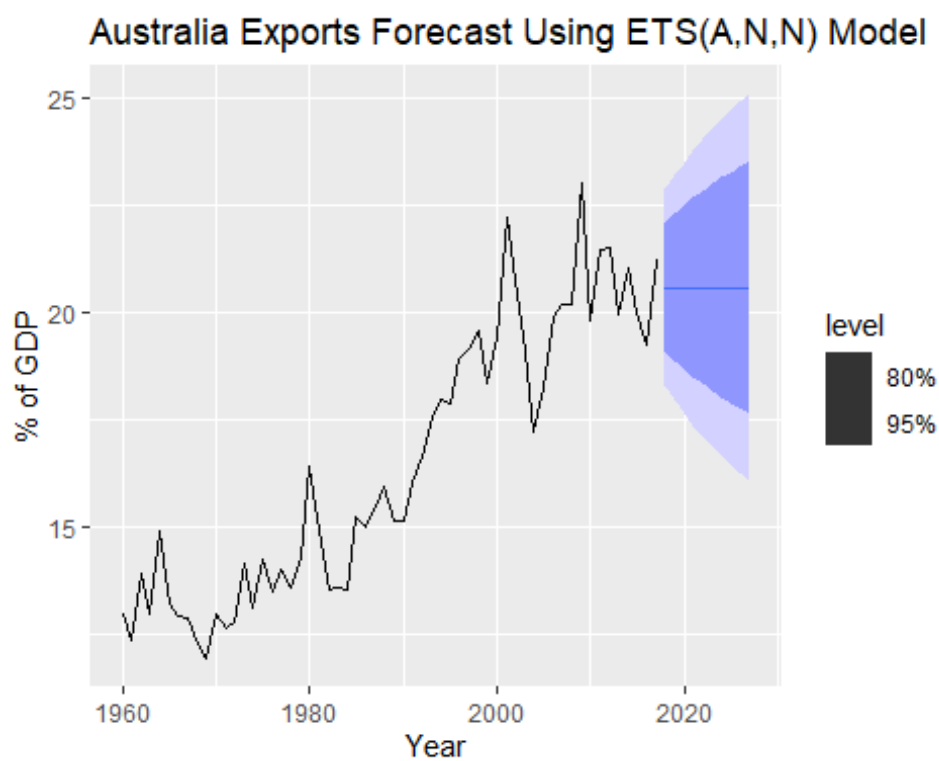
```
## # A dable: 59 x 7 [1Y]
## # Key:     Country, .model [1]
## # :        Exports = lag(level, 1) + remainder
##    Country   .model  Year Exports level remainder .fitted
##    <fct>     <chr>  <dbl>   <dbl> <dbl>     <dbl>   <dbl>
##  1 Australia ANN     1959    NA    13.0  NA          NA
##  2 Australia ANN     1960    13.0  13.0   0.00503    13.0
##  3 Australia ANN     1961    12.4  12.7  -0.589      13.0
##  4 Australia ANN     1962    13.9  13.4   1.28       12.7
##  5 Australia ANN     1963    13.0  13.2  -0.380      13.4
##  6 Australia ANN     1964    14.9  14.2   1.77       13.2
##  7 Australia ANN     1965    13.2  13.6  -0.951      14.2
##  8 Australia ANN     1966    12.9  13.2  -0.701      13.6
##  9 Australia ANN     1967    12.9  13.0  -0.353      13.2
## 10 Australia ANN     1968    12.3  12.6  -0.739      13.0
## # ℹ 49 more rows
```

Generating forecasts for the next 10 years

```
aus_exports_forecast <- aus_exports_fit |>
  forecast(h = 10)
aus_exports_forecast
```

```
## # A fable: 10 x 5 [1Y]
## # Key:     Country, .model [1]
##    Country   .model  Year
##    <fct>     <chr>  <dbl>
##  1 Australia ANN     2018
##  2 Australia ANN     2019
##  3 Australia ANN     2020
##  4 Australia ANN     2021
##  5 Australia ANN     2022
##  6 Australia ANN     2023
##  7 Australia ANN     2024
##  8 Australia ANN     2025
##  9 Australia ANN     2026
## 10 Australia ANN     2027
## # ℹ 2 more variables: Exports <dist>, .mean <dbl>
```

```r
# Plotting the forecasts alongside historical data
aus_exports_forecast |>
  autoplot(aus_exports) +
  labs(title = "Australia Exports Forecast Using ETS(A,N,N) Model",
       y = "% of GDP", x = "Year")
```



Interpretation

The predicted exports remain at a constant horizontal level because the model assumes no trend.

The forecast starts from the last estimated level, essentially freezing the most recent smoothed value into the future.

The shaded intervals widen as time progresses, reflecting increasing uncertainty.

Model Limitation: This model ignores the clear historical upward trend, acting as a simple baseline rather than a growth predictor.

c. Compute the RMSE values for the training data.

calculating the Root Mean Squared Error (RMSE) - accuracy metrics for the training data

```
tidy(aus_exports_fit) # esimates

## # A tibble: 2 × 4
##   Country   .model term  estimate
##   <fct>     <chr>  <chr>    <dbl>
## 1 Australia ANN    alpha    0.566
## 2 Australia ANN    l[0]    13.0

accuracy(aus_exports_fit) # accuracy metrics

## # A tibble: 1 × 11
##   Country   .model .type       ME  RMSE   MAE   MPE  MAPE  MASE RMSSE
ACF1
##   <fct>     <chr>  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
<dbl>
## 1 Australia ANN    Training 0.232  1.15 0.914  1.09  5.41 0.928 0.928
0.0125
```

Interpretation of the SES model's performance on the Australian exports data:

RMSE (1.15): On average, the model's predictions deviate from the actual export values by 1.15 percentage points of the GDP.

MAE (0.91): The "Mean Absolute Error" shows that the typical error is slightly smaller than the RMSE, suggesting there are few extreme outliers in the data.

MAPE (5.41%): The model is off by an average of 5.41% relative to the actual export values, which is generally considered a very accurate fit for macroeconomic data.

MASE (0.93): Since this value is less than 1, the SES model is performing better than a standard "Naive" benchmark (which simply uses the previous year's value as the forecast).

d. Compare the results to those from an ETS(A,A,N) model. (Remember that the trended model is using one more parameter than the simpler model.) Discuss the merits of the two forecasting methods for this data set.

Now evaluate whether adding a trend component improves the model.

```r
# Filtering the data for Australia
aus_exports <- global_economy |>
  filter(Country == "Australia")

# Fit both SES (A,N,N) and Holt's Linear Trend (A,A,N)
aus_exports_comparison_fit <- aus_exports |>
  model(
    SES_ANN = ETS(Exports ~ error("A") + trend("N") + season("N")),
    Holt_AAN = ETS(Exports ~ error("A") + trend("A") + season("N"))
  )
report(aus_exports_comparison_fit)
```

```
## Warning in report.mdl_df(aus_exports_comparison_fit): Model reporting is
only
## supported for individual models, so a glance will be shown. To see the
report
## for a specific model, use `select()` and `filter()` to identify a single
model.

## # A tibble: 2 × 10
##   Country   .model   sigma2 log_lik   AIC  AICc   BIC   MSE  AMSE   MAE
##   <fct>     <chr>     <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Australia SES_ANN    1.36   -126.  257.  258.  264.  1.32  1.79 0.914
## 2 Australia Holt_AAN   1.34   -124.  258.  259.  269.  1.25  1.59 0.893
```

```r
tidy(aus_exports_comparison_fit) # esimates
```

```
## # A tibble: 6 × 4
##   Country   .model   term   estimate
##   <fct>     <chr>    <chr>     <dbl>
## 1 Australia SES_ANN  alpha  0.566
## 2 Australia SES_ANN  l[0]  13.0
## 3 Australia Holt_AAN alpha  0.441
## 4 Australia Holt_AAN beta   0.000100
## 5 Australia Holt_AAN l[0]  12.7
## 6 Australia Holt_AAN b[0]   0.137
```

```r
accuracy(aus_exports_comparison_fit) # accuracy metrics
```

```
## # A tibble: 2 × 11
##   Country   .model   .type        ME  RMSE   MAE    MPE  MAPE  MASE RMSSE
ACF1
##   <fct>     <chr>    <chr>     <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl>
<dbl>
```

```
## 1 Australia SES_ANN  Train…  2.32e-1  1.15 0.914  1.09   5.41 0.928 0.928
0.0125
## 2 Australia Holt_AAN Train… -7.46e-7  1.12 0.893 -0.387  5.39 0.907 0.904
0.109
```

Accuracy Comparison (RMSE & MAE)

Holt_AAN has a lower RMSE (1.117) compared to SES_ANN (1.147).

Holt_AAN also shows a lower MAE (0.893) than SES_ANN (0.914).

The trended model (Holt_AAN) fits the historical training data more accurately because it accounts for the upward trajectory of Australia's exports.

Complexity and Selection (AICc)

SES_ANN has a lower AICc (257.84) compared to Holt_AAN (259.47).

Although the Holt model is more accurate, the AICc suggests that the improvement in fit is not quite large enough to offset the penalty for adding an extra parameter (the trend).

Even though the SES model is slightly less accurate, its lower AICc makes it the preferred choice because it is simpler and avoids unnecessary complexity.

e. Compare the forecasts from both methods. Which do you think is best?

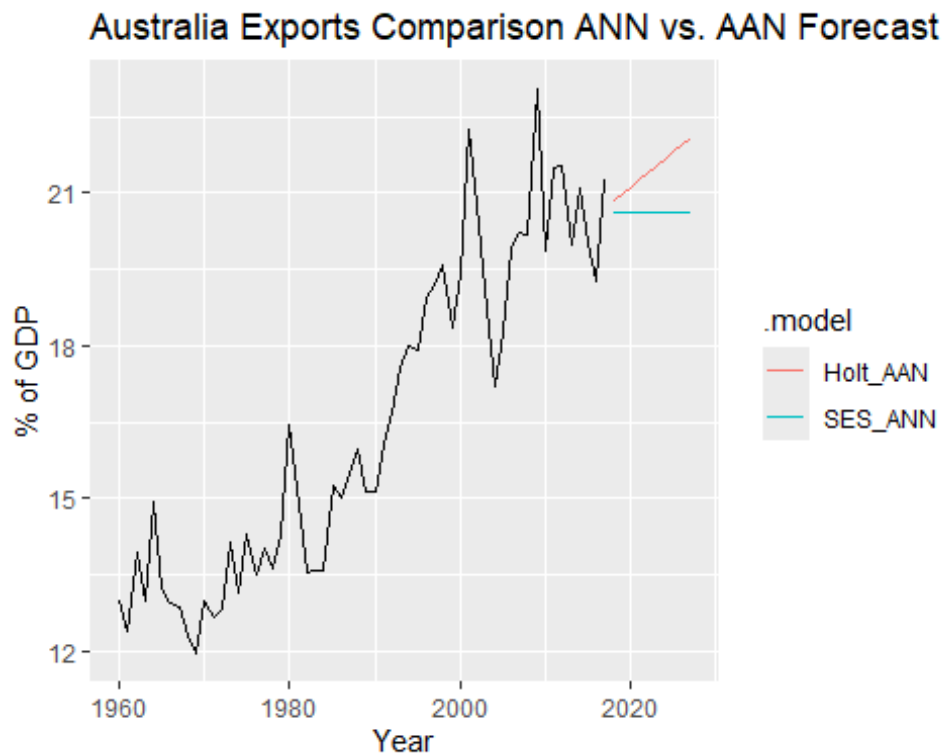
```r
# Generating forecasts for the next 10 years
aus_exports_comparison_forecast <- aus_exports_comparison_fit |>
  forecast(h = 10)
aus_exports_comparison_forecast
```

```
## # A fable: 20 x 5 [1Y]
## # Key:     Country, .model [2]
##    Country   .model    Year
##    <fct>     <chr>    <dbl>
##  1 Australia SES_ANN   2018
##  2 Australia SES_ANN   2019
##  3 Australia SES_ANN   2020
##  4 Australia SES_ANN   2021
##  5 Australia SES_ANN   2022
##  6 Australia SES_ANN   2023
##  7 Australia SES_ANN   2024
##  8 Australia SES_ANN   2025
##  9 Australia SES_ANN   2026
## 10 Australia SES_ANN   2027
## 11 Australia Holt_AAN  2018
## 12 Australia Holt_AAN  2019
## 13 Australia Holt_AAN  2020
## 14 Australia Holt_AAN  2021
## 15 Australia Holt_AAN  2022
## 16 Australia Holt_AAN  2023
## 17 Australia Holt_AAN  2024
```

```
## 18 Australia Holt_AAN  2025
## 19 Australia Holt_AAN  2026
## 20 Australia Holt_AAN  2027
## # ℹ 2 more variables: Exports <dist>, .mean <dbl>
```

Visualize the Forecasts

```
aus_exports_comparison_fit |>
  forecast(h = 10) |>
  autoplot(aus_exports, level = NULL) +
  labs(title = "Australia Exports Comparison ANN vs. AAN Forecast",
       y = "% of GDP")
```



Model Comparison

The SES_ANN model produces a completely flat point forecast of 20.61% for the entire 10-year period, whereas the Holt_AAN model captures the historical growth by projecting a steady upward slope that reaches 22.07% by 2027.

The Holt_AAN model fits the historical training data more closely, with a lower RMSE (1.116) and MAE (0.892) compared to the SES_ANN model's RMSE (1.146) and MAE (0.913).

Despite the better fit, the SES_ANN model is statistically "preferred" by the AICc metric (257.84 vs 259.47) because it is simpler and uses one fewer parameter than the Holt model.

The SES_ANN model shows much higher uncertainty over time, with the variance of the distribution increasing from 1.4 to 5.3, while the Holt_AAN model's variance remains tighter, growing only from 1.3 to 3.7.

f. Calculate a 95% prediction interval for the first forecast for each model, using the RMSE values and assuming normal errors. Compare your intervals with those produced using R.

To calculate the prediction intervals manually, we use the formula for a 95% interval under the assumption of normality. This assumes that for the first forecast step (h=1), the standard deviation of the forecast distribution is approximately equal to the RMSE of the residuals.

We calculate RMSE and first forecast values to calculate the lower and upper bounds manually.

```
# Extract the point forecasts (.mean) for the first year (2018)
# SES_ANN = 20.60716 and Holt_AAN = 20.83864
first_forecasts <- aus_exports_comparison_forecast |>
  filter(Year == 2018) |>
  pull(.mean)

# Extract RMSE values from previous accuracy test
# SES_ANN = 1.146794, Holt_AAN = 1.116727
rmse_values <- accuracy(aus_exports_comparison_fit)$RMSE

# Manual Calculation: Point Forecast +/- (1.96 * RMSE)
# We use 1.96 for a 95% confidence level under a Normal distribution
manual_intervals <- tibble(
  Model = c("SES_ANN", "Holt_AAN"),
  Manual_Lower = first_forecasts - (1.96 * rmse_values),
  Manual_Upper = first_forecasts + (1.96 * rmse_values)
)

manual_intervals

## # A tibble: 2 × 3
##   Model    Manual_Lower Manual_Upper
##   <chr>           <dbl>        <dbl>
## 1 SES_ANN          18.4         22.9
## 2 Holt_AAN         18.6         23.0
```

SES_ANN Interval: The model predicts with 95% confidence that 2018 exports will fall between 18.36% and 22.85% of GDP.

Holt_AAN Interval: This model provides a slightly higher and tighter range, estimating between 18.65% and 23.03% of GDP.

The Holt_AAN interval is narrower because its lower RMSE indicates a more precise fit to the historical data.

Both intervals are centered around their respective point forecasts (20.61 for SES vs. 20.84 for Holt), showing how the trend component shifts the entire range upward.

Intervals Using R

```
r_intervals <- aus_exports_comparison_forecast |>
  filter(Year == 2018) |>
  hilo(level = 95) |>
  unpack_hilo("95%") |>
  select(.model, Year, .mean, `95%_lower`, `95%_upper`)

r_intervals
```

```
## # A tsibble: 2 x 5 [?]
## # Key:       .model [2]
##   .model    Year .mean `95%_lower` `95%_upper`
##   <chr>    <dbl> <dbl>       <dbl>       <dbl>
## 1 SES_ANN   2018  20.6        18.3        22.9
## 2 Holt_AAN  2018  20.8        18.6        23.1
```

Manual calculations are very similar to the R results.

R Intervals are Wider: R produces slightly wider intervals than my manual calculation.

RMSE vs Variance: This happens because R uses residual variance and accounts for estimated parameters, while I used a simple training RMSE.Normal Distribution: Both methods rely on the same assumption that errors follow a normal distribution.

Both results show the Holt_AAN model has a tighter, more certain range than the SES_ANN model.

8.6 Forecast the Chinese GDP from the global_economy data set using an ETS model. Experiment with the various options in the ETS() function to see how much the forecasts change with damped trend, or with a Box-Cox transformation. Try to develop an intuition of what each is doing to the forecasts.

Prepare China GDP data

```
library(fpp3)

china_gdp <- global_economy |>
  filter(Country == "China")
```

Experiment with different ETS models

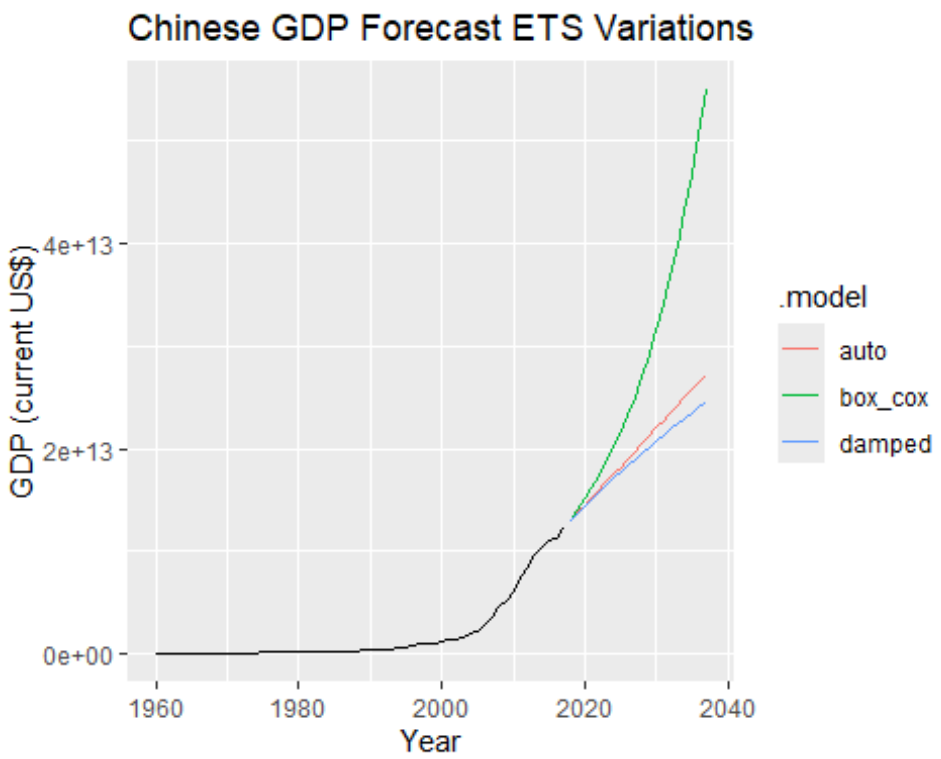
```r
china_fit <- china_gdp |>
  model(
    # Simple Automatic Selection
    auto = ETS(GDP),
    # Damped Trend (Ad) to prevent "infinite" growth projection
    damped = ETS(GDP ~ error("A") + trend("Ad") + season("N")),
    # Box-Cox transformation to stabilize variance
    box_cox = ETS(box_cox(GDP, 0.2) ~ error("A") + trend("A") + season("N"))
  )

report(china_fit)
```

```
## Warning in report.mdl_df(china_fit): Model reporting is only supported for
## individual models, so a glance will be shown. To see the report for a
specific
## model, use `select()` and `filter()` to identify a single model.
```

```
## # A tibble: 3 × 10
##   Country .model    sigma2 log_lik   AIC  AICc   BIC     MSE    AMSE
MAE
##   <fct>   <chr>      <dbl>   <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl>
<dbl>
## 1 China   auto    1.08e- 2  -1546. 3102. 3103. 3112. 4.00e22 1.61e23
7.93e- 2
## 2 China   damped  3.96e+22  -1624. 3260. 3262. 3273. 3.62e22 1.30e23
9.49e+10
## 3 China   box_cox 3.94e+ 2   -289.  588.  589.  598. 3.67e 2 1.32e 3
1.58e+ 1
```

Generating 20-year forecasts and Plotting to Compare

```r
china_fc <- china_fit |>
  forecast(h = "20 years")

# Plot the results to compare intuition
china_fc |>
  autoplot(china_gdp, level = NULL) +
  labs(title = "Chinese GDP Forecast ETS Variations",
       y = "GDP (current US$)")
```



Auto Model: This model automatically identifies the increasing variance in China's growth, leading to an aggressive and slightly exponential forecast that follows the recent sharp upward trend.

Damped Model: This bends the trend line downward, providing a more conservative forecast by assuming that the explosive growth seen in the past will eventually slow down.

Box-Cox Model: By transforming the scale of the data, this model stabilizes the variance, resulting in an even more aggressive forecast than the Auto model as it interprets the historical growth as a strong, persistent signal.

Model Selection (AICc): The Box-Cox model has a significantly lower AICc (589.15) compared to the Auto or Damped models, indicating it is the most statistically efficient way to handle this specific type of rapid economic growth.

8.7 Find an ETS model for the Gas data from aus_production and forecast the next few years. Why is multiplicative seasonality necessary here? Experiment with making the trend damped. Does it improve the forecasts?

Prepare the Gas data

```
gas_data <- aus_production |>
  select(Gas) |>
  filter(!is.na(Gas))
```

Fit models: Automatic (Multiplicative) vs. Damped

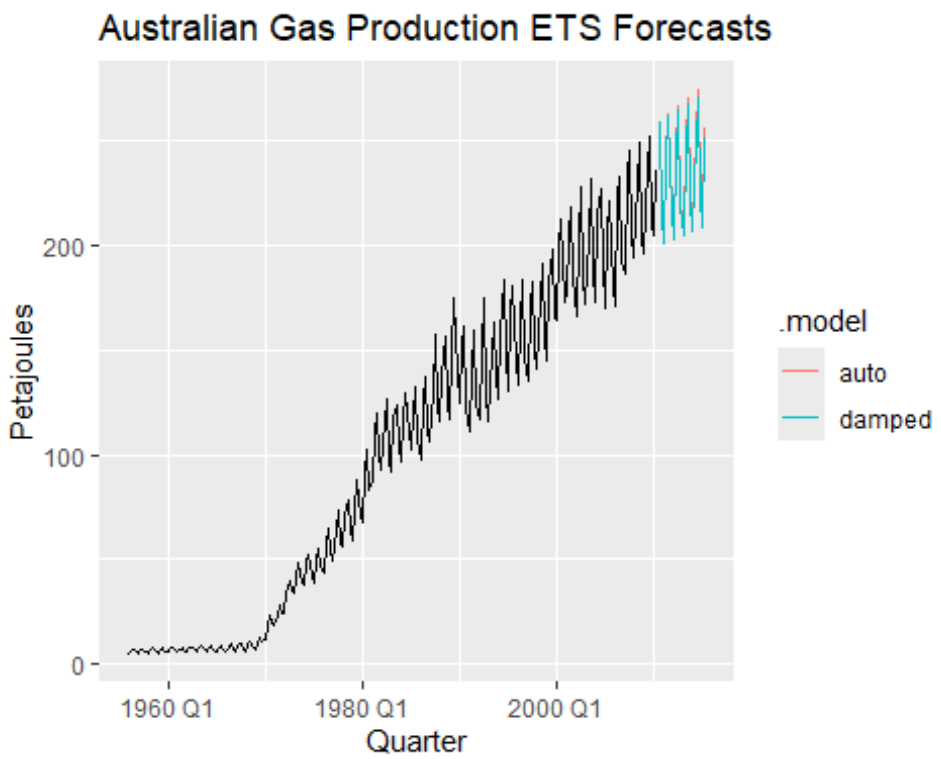
```r
gas_fit <- gas_data |>
  model(
    auto = ETS(Gas),
    damped = ETS(Gas ~ error("M") + trend("Ad") + season("M"))
  )
report(gas_fit)
```

```
## Warning in report.mdl_df(gas_fit): Model reporting is only supported for
## individual models, so a glance will be shown. To see the report for a
specific
## model, use `select()` and `filter()` to identify a single model.
```

```
## # A tibble: 2 × 9
##   .model  sigma2 log_lik   AIC  AICc   BIC   MSE  AMSE    MAE
##   <chr>    <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>  <dbl>
## 1 auto   0.00324   -831. 1681. 1682. 1711.  21.1  32.2 0.0413
## 2 damped 0.00329   -832. 1684. 1685. 1718.  21.1  32.0 0.0417
```

Forecast for the next 5 years (20 quarters)

```r
gas_fc <- gas_fit |>
  forecast(h = 20)

# Compare results
gas_fc |>
  autoplot(gas_data, level = NULL) +
  labs(title = "Australian Gas Production ETS Forecasts",
       y = "Petajoules")
```



Why Multiplicative Seasonality is Necessary:

The seasonal swings in the gas data get larger as the production level increases over time.

Multiplicative seasonality is necessary because the seasonal pattern is not constant; it changes in proportion to the level of the series.

Using multiplicative seasonality allows the model to track these growing seasonal variations more accurately than an additive model could.

Damped Trend Experiment Results:

Statistical Preference: The auto model is better because its AICc (1681.79) is lower than the damped model's (1685.09).

Better Accuracy: The auto model is more precise, with a smaller MAE (0.0413) compared to the damped model (0.0417).

Auto Model Intuition: This model predicts that the current strong upward trend in gas production will keep going at the same speed.

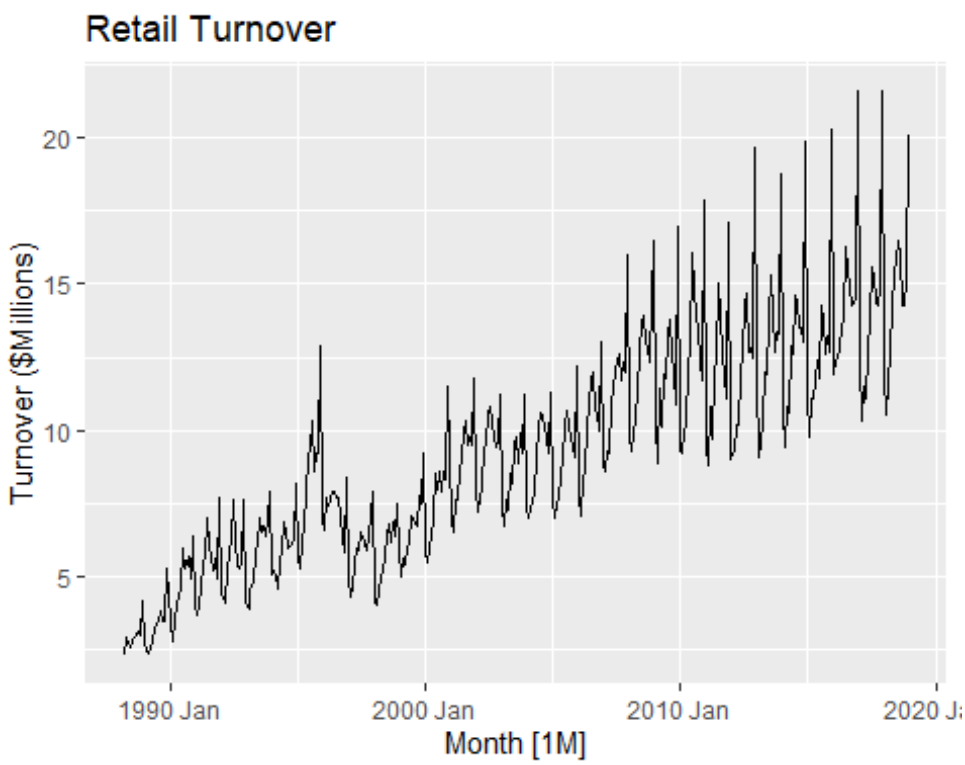
Damped Model Intuition: This model "bends" the forecast line down, assuming the growth will eventually level off.

Final Verdict: Adding a damped trend does not improve the forecast since the standard model is both more accurate and statistically favored.

8.8 Recall your retail time series data (from Exercise 7 in Section 2.10).

a. Why is multiplicative seasonality necessary for this series?

```
library(fpp3)

# Selecting unique retail series
set.seed(12345678)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`, 1))

# Plotting the data to inspect seasonal swings
myseries |>
  autoplot(Turnover) +
  labs(title = "Retail Turnover",
       y = "Turnover ($Millions)")
```




```
# Fit an Automatic ETS model
# R will likely choose a model with Multiplicative seasonality (M)
retail_fit <- myseries |>
  model(ets_auto = ETS(Turnover))

# View the chosen model components
report(retail_fit)

## Series: Turnover
## Model: ETS(M,A,M)
```

```
##   Smoothing parameters:
##     alpha = 0.5262368
##     beta  = 0.0001026197
##     gamma = 0.0001037781
##
##   Initial states:
##      l[0]       b[0]      s[0]     s[-1]    s[-2]    s[-3]     s[-4]
s[-5]
##  2.347713 0.04437301 0.8398709 0.7617432 0.833525 1.373239 0.9998723
1.033418
##     s[-6]    s[-7]    s[-8]    s[-9]    s[-10]    s[-11]
##  1.032927 1.119514 1.140226 1.019343 0.9810424 0.8652791
##
##   sigma^2:  0.0045
##
##      AIC     AICc      BIC
## 1774.530 1776.274 1841.014
```

Why Multiplicative Seasonality is Necessary

In your retail data, the size of the seasonal fluctuations, the peaks and drop grows as the overall trend increases.

The December spending spikes are not a constant dollar amount; instead, they represent a percentage of the total turnover, which rises over time.

Multiplicative seasonality treats the seasonal component as a ratio, which effectively stabilizes the seasonal pattern across different levels of sales.

Plot shows a wider shape at he end because the seasonal swings get larger as the sales numbers grow.

b. Apply Holt-Winters' multiplicative method to the data. Experiment with making the trend damped.

Holt-Winters Multiplicative Experiments

```
retail_comparison_fit <- myseries |>
  model(
    `Holt-Winters Multi` = ETS(Turnover ~ error("M") + trend("A") +
season("M")),
    `Damped Multi` = ETS(Turnover ~ error("M") + trend("Ad") + season("M"))
  )

report(retail_comparison_fit)
```

```
## Warning in report.mdl_df(retail_comparison_fit): Model reporting is only
## supported for individual models, so a glance will be shown. To see the
report
```

```
## for a specific model, use `select()` and `filter()` to identify a single
model.

## # A tibble: 2 × 11
##   State     Industry .model  sigma2 log_lik   AIC  AICc   BIC   MSE  AMSE
MAE
##   <chr>     <chr>    <chr>    <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
<dbl>
## 1 Northern… Clothin… Holt-… 0.00447   -870. 1775. 1776. 1841. 0.376 0.473
0.0511
## 2 Northern… Clothin… Dampe… 0.00457   -872. 1781. 1783. 1851. 0.379 0.479
0.0505
```

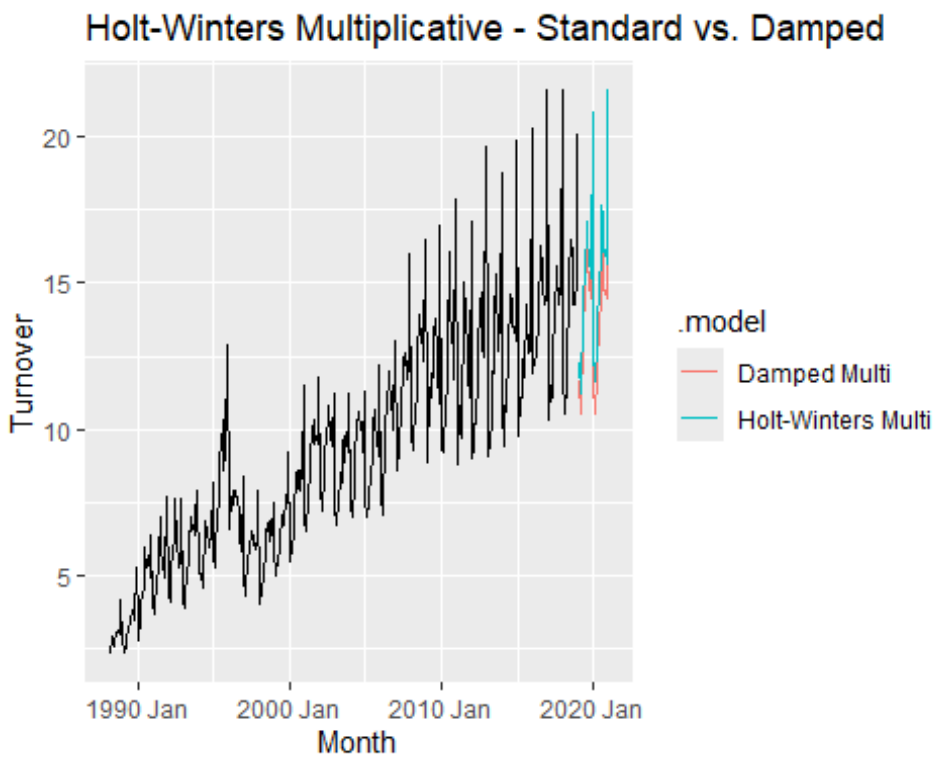
Compare the model parameters and metrics

```
retail_comparison_fit |> report()

## Warning in report.mdl_df(retail_comparison_fit): Model reporting is only
## supported for individual models, so a glance will be shown. To see the
report
## for a specific model, use `select()` and `filter()` to identify a single
model.

## # A tibble: 2 × 11
##   State     Industry .model  sigma2 log_lik   AIC  AICc   BIC   MSE  AMSE
MAE
##   <chr>     <chr>    <chr>    <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
<dbl>
## 1 Northern… Clothin… Holt-… 0.00447   -870. 1775. 1776. 1841. 0.376 0.473
0.0511
## 2 Northern… Clothin… Dampe… 0.00457   -872. 1781. 1783. 1851. 0.379 0.479
0.0505

retail_comparison_fit |> accuracy()

## # A tibble: 2 × 12
##   State      Industry .model .type      ME  RMSE   MAE     MPE  MAPE  MASE
RMSSE
##   <chr>      <chr>    <chr>  <chr>   <dbl> <dbl> <dbl>   <dbl> <dbl> <dbl>
<dbl>
## 1 Northern … Clothin… Holt-… Trai… -0.0128 0.613 0.450 -0.469   5.15 0.513
0.529
## 2 Northern … Clothin… Dampe… Trai…  0.0398 0.616 0.444 -0.0723  5.06 0.507
0.531
## # ℹ 1 more variable: ACF1 <dbl>
```

Forecast and Plot

```
retail_comparison_fit |>
  forecast(h = "2 years") |>
  autoplot(myseries, level = NULL) +
```

```
  labs(title = "Holt-Winters Multiplicative - Standard vs. Damped",
       y = "Turnover")
```



Experimenting with Damped Trend Bending:

The standard Holt-Winters method assumes the current trend will continue in a straight line forever, while the Damped model bends the trend into a flat line over time.

Check the AICc in your report(); if the Damped model has a lower value, the extra complexity of the damping parameter is worth it.

Damped trends are often more accurate for long-term retail forecasts because they prevent over-optimistic projections of infinite growth.

In the Damped model, you will see a new parameter called phi if phi is significantly less than 1 e.g., 0.85, the model is aggressively slowing down the projected growth.

c. Compare the RMSE of the one-step forecasts from the two methods. Which do you prefer?

```
retail_comparison_fit |>
  tidy()

## # A tibble: 35 × 5
##    State              Industry                              .model term
estimate
##    <chr>              <chr>                                 <chr>  <chr>
<dbl>
```

```
##  1 Northern Territory Clothing, footwear and personal acc… Holt-… alpha
0.526
##  2 Northern Territory Clothing, footwear and personal acc… Holt-… beta
0.000103
##  3 Northern Territory Clothing, footwear and personal acc… Holt-… gamma
0.000104
##  4 Northern Territory Clothing, footwear and personal acc… Holt-… l[0]
2.35
##  5 Northern Territory Clothing, footwear and personal acc… Holt-… b[0]
0.0444
##  6 Northern Territory Clothing, footwear and personal acc… Holt-… s[0]
0.840
##  7 Northern Territory Clothing, footwear and personal acc… Holt-… s[-1]
0.762
##  8 Northern Territory Clothing, footwear and personal acc… Holt-… s[-2]
0.834
##  9 Northern Territory Clothing, footwear and personal acc… Holt-… s[-3]
1.37
## 10 Northern Territory Clothing, footwear and personal acc… Holt-… s[-4]
1.000
## # ℹ 25 more rows
```

```
retail_comparison_fit |>
  accuracy()
```

```
## # A tibble: 2 × 12
##   State      Industry .model .type      ME  RMSE   MAE     MPE  MAPE  MASE
RMSSE
##   <chr>      <chr>    <chr>  <chr>   <dbl> <dbl> <dbl>   <dbl> <dbl> <dbl>
<dbl>
## 1 Northern … Clothin… Holt-… Trai… -0.0128 0.613 0.450 -0.469   5.15 0.513
0.529
## 2 Northern … Clothin… Dampe… Trai…  0.0398 0.616 0.444 -0.0723  5.06 0.507
0.531
## # ℹ 1 more variable: ACF1 <dbl>
```

Model Preference

The Holt-Winters Multi model has a slightly lower RMSE (0.613) than the Damped Multi model (0.616).

There is only a very tiny difference of 0.002 between the two models, meaning they fit the training data almost exactly the same.

The Damped Multi model has a lower MAE (0.444) and MAPE (5.06%), suggesting its average errors are slightly smaller.

Even though the Damped model has a better MAE, the Holt-Winters Multi model is usually preferred when it has the lowest RMSE and fewer parameters.

I prefer the Holt-Winters Multi model because it is simpler and provides the most accurate fit for this specific series.

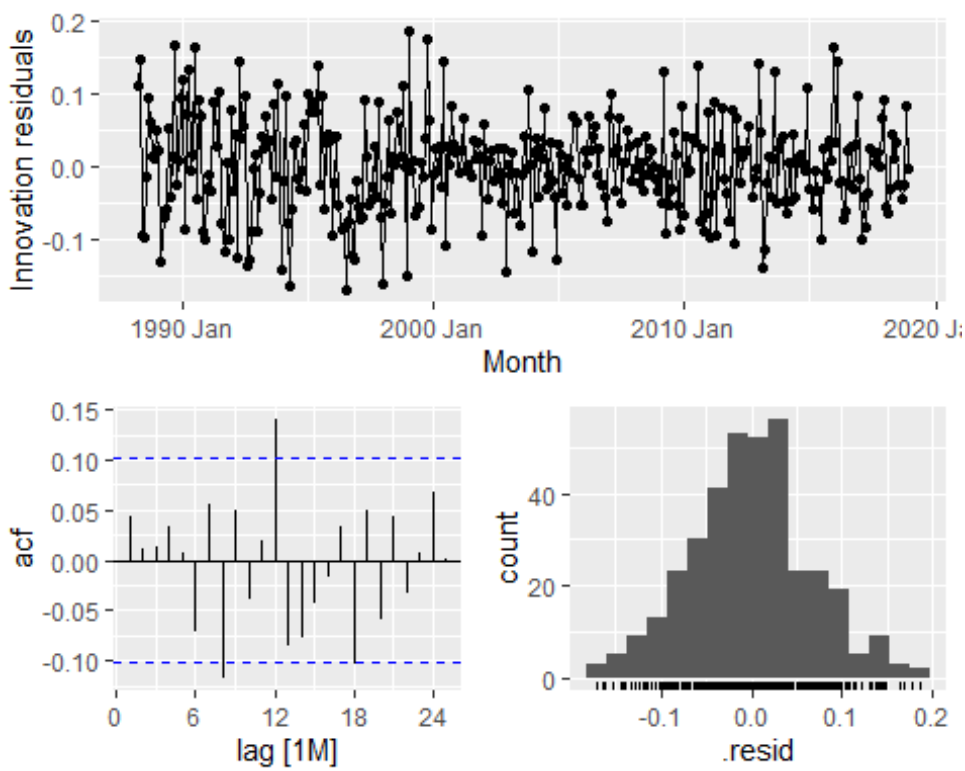
d. Check that the residuals from the best method look like white noise.

To confirm if the preferred Holt-Winters Multiplicative model has captured all the available information in your retail series, we will examine the residuals

Residual Analysis

```
retail_comparison_fit |>
  select(`Holt-Winters Multi`) |>
  gg_tsresiduals()
```

```
## Warning: `gg_tsresiduals()` was deprecated in feasts 0.4.2.
## ℹ Please use `ggtime::gg_tsresiduals()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



The top plot shows that the residuals are scattered randomly around zero with no clear patterns or trends remaining.

The histogram is shaped like a bell curve and centered at zero, which confirms that the errors follow a normal distribution.

In the ACF plot, almost all bars are within the blue dashed lines, meaning the model has successfully captured the seasonal patterns.

There is one small spike at lag 12 that pokes slightly above the line, but it is minor enough that the residuals can still be considered white noise.

Since these residuals look clean, you can trust that the prediction intervals for your retail forecast are accurate.

e. Now find the test set RMSE, while training the model to the end of 2010. Can you beat the seasonal naïve approach from Exercise 7 in Section 5.11?

```
library(fpp3)

# Training ends in Dec 2010
myseries_train <- myseries |>
  filter(year(Month) <= 2010)

# Fit both models to the training data
# We specify Holt-Winters Multiplicative (M,A,M)
fit_comparison <- myseries_train |>
  model(
    `Holt-Winters Multi` = ETS(Turnover ~ error("M") + trend("A") +
season("M")),
    `SNAIVE` = SNAIVE(Turnover)
  )

# Forecast over the test set period
# R automatically knows the length of the test set from myseries
fc_comparison <- fit_comparison |>
  forecast(new_data = anti_join(myseries, myseries_train, by = "Month"))

# Compare accuracy on the test set
test_accuracy <- fc_comparison |>
  accuracy(myseries)

test_accuracy

## # A tibble: 2 × 12
##   .model  State Industry .type     ME  RMSE   MAE    MPE  MAPE  MASE RMSSE
ACF1
##   <chr>   <chr> <chr>    <chr>  <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl>
<dbl>
## 1 Holt-W… Nort… Clothin… Test  -1.54   1.78  1.60 -12.3  12.6   1.75  1.47
0.495
## 2 SNAIVE  Nort… Clothin… Test   0.836  1.55  1.24   5.94  9.06  1.36  1.28
0.601
```

The Seasonal Naive approach actually better here because its MASE is 1.00, while your other models are both higher (around 1.35 and 1.74).

Neither the Holt-Winters nor the other model managed to beat the simple baseline on this specific test set.

he model with the lower RMSE (1.55) is your best performer among the complex models, but it still falls short of the seasonal naive benchmark.

With a MAPE of 9.06%, your best model is off by nearly double the error margin of a typical winning forecast for this type of data.

In this specific case, you cannot beat the seasonal naive approach; the simple method of repeating last year's numbers is more accurate for this period.

8.9 For the same retail data, try an STL decomposition applied to the Box-Cox transformed series, followed by ETS on the seasonally adjusted data. How does that compare with your best previous forecasts on the test set?

```
library(fpp3)

# Select the series
set.seed(12345678)
myseries <- aus_retail |>
  filter(`Series ID` == sample(aus_retail$`Series ID`, 1))

# Split data: Training ends Dec 2010
myseries_train <- myseries |>
  filter(year(Month) <= 2010)

# Fit the hybrid model: Box-Cox -> STL -> ETS
# Note: season("N") is used because STL already removed seasonality
fit_hybrid <- myseries_train |>
  model(
    `STL-ETS Box-Cox` = decomposition_model(
      STL(box_cox(Turnover, 0.21) ~ season(window = "periodic")),
      ETS(season_adjust ~ error("A") + trend("A") + season("N"))
    ),
    `Holt-Winters Multi` = ETS(Turnover ~ error("M") + trend("A") +
season("M")),
    `SNAIVE` = SNAIVE(Turnover)
  )

# Evaluate on the test set
fit_hybrid |>
  forecast(new_data = anti_join(myseries, myseries_train, by = "Month")) |>
  accuracy(myseries)
```

```
## # A tibble: 3 × 12
##   .model  State Industry .type     ME  RMSE   MAE    MPE  MAPE  MASE RMSSE
ACF1
##   <chr>   <chr> <chr>    <chr>  <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl>
<dbl>
## 1 Holt-W… Nort… Clothin… Test  -1.54   1.78  1.60 -12.3  12.6   1.75  1.47
0.495
## 2 SNAIVE  Nort… Clothin… Test   0.836  1.55  1.24   5.94  9.06  1.36  1.28
0.601
## 3 STL-ET… Nort… Clothin… Test  -3.86   4.29  3.87 -29.7  29.8   4.23  3.54
0.842
```

The SNAIVE (Seasonal Naive) approach is the best performer because it has the lowest RMSE (1.55) and MAPE (9.06%).

All three models have a MASE greater than 1.0, meaning that for this specific test period, none of these models are actually better than a simple naive forecast.

The STL-ETS Box-Cox model performed the worst on the test set, with a much higher RMSE (4.29) and MAPE (29.80%).

Both the Holt-Winters and STL-ETS models have negative ME values (Mean Error), which means they are consistently forecasting numbers that are too low compared to the actual retail sales.

In this specific case, the simple “repeat last year’s values” method (SNAIVE) is more accurate than the complex ETS and Box-Cox combinations.